Assessment completed!

DEMONSTRATED PROFICIENCY

Advanced

80% score

TIME TO COMPLETE

00:34:18

DATE COMPLETED

05/12/2026

Due February 15, 2026

SECURE CODING

80%

Strengths

- ✓ Missing Authorization
- ✓ Risky Cryptographic Algorithm
- ✓ Integer Overflow or Wraparound
- ✓ Improper Authentication
- ✓ Improper Access Control
- ✓ Command Injection
- ✓ SQL Injection
- ✓ Insecure Design
- ✓ Logging and Monitoring Failures
- ✓ Server-Side Request Forgery
- ✓ Software and Data Integrity Failure
- ✓ Vulnerable and Outdated Components

Growth Opportunities

- ⚠ Race Condition
- ⚠ Improper Input Validation
- ⚠ Security Misconfiguration

QUESTION 1: (PYTHON) AUTHENTICATION AND AUTHORIZATION ISSUES | MISSING AUTHORIZATION

While reviewing a SaaS project management platform's user management system, you discover code handling team member access to project resources. The system is used by thousands of organizations to manage sensitive project data. Identify the block of code that could allow unauthorized access to project resources.

Your answer

D, which is lines 57-64

Correct answer

D, which is lines 57-64.

```
1    from flask import Flask, request, session, jsonify
2    from flask_sqlalchemy import SQLAlchemy
3    from werkzeug.security import check_password_hash
4    from datetime import datetime
5    import logging
6
7    app = Flask(__name__)
8    db = SQLAlchemy(app)
9    logger = logging.getLogger(__name__)
10
11
A   12  class User(db.Model):
13      id = db.Column(db.Integer, primary_key=True)
14      email = db.Column(db.String(120), unique=True, nullable=False)
15      password_hash = db.Column(db.String(256), nullable=False)
16      organization_id = db.Column(db.Integer, db.ForeignKey('organization.id'))
17      role = db.Column(db.String(20), nullable=False)
18      last_login = db.Column(db.DateTime)
```

```
19
20
B 21  class Project(db.Model):
22        id = db.Column(db.Integer, primary_key=True)
23        name = db.Column(db.String(100), nullable=False)
24        organization_id = db.Column(db.Integer, db.ForeignKey('organization.id'))
25        created_at = db.Column(db.DateTime, default=datetime.utcnow)
26        data = db.Column(db.JSON)
27
28
C 29  @app.route('/api/v1/auth/login', methods=['POST'])
30    def login():
31        email = request.json.get('email')
32        password = request.json.get('password')
33
34        if not email or not password:
35            return jsonify({'error': 'Missing credentials'}), 400
36
37        user = User.query.filter_by(email=email).first()
38        if user and check_password_hash(user.password_hash, password):
39            session['user_id'] = user.id
40            user.last_login = datetime.utcnow()
41            db.session.commit()
42            logger.info(f"User {email} logged in successfully")
43            return jsonify({'status': 'success'})
44
45        logger.warning(f"Failed login attempt for {email}")
46        return jsonify({'error': 'Invalid credentials'}), 401
47
```

```
48
49  @app.route('/api/v1/auth/logout', methods=['POST'])
50  def logout():
51      user_id = session.pop('user_id', None)
52      if user_id:
53          logger.info(f"User {user_id} logged out")
54      return jsonify({'status': 'success'})
55
56
D 57  @app.route('/api/v1/projects/<int:project_id>/data', methods=['GET'])
58  def get_project_data(project_id):
59      project = Project.query.get_or_404(project_id)
60      return jsonify({
61          'id': project.id,
62          'name': project.name,
63          'data': project.data
64      })
65
```

Provide question feedback

QUESTION 2: (PYTHON) CONCURRENCY ISSUES | CONCURRENT EXECUTION USING SHARED RESOURCE WITH IMPROPER SYNCHRONIZATION ('RACE CONDITION')

During a security audit of a high-volume e-commerce platform's user authentication system, you discover code that handles concurrent user sessions and account creation. The system processes thousands of registrations per minute during peak sales events. Which line introduces a race condition vulnerability that could lead to duplicate usernames or data inconsistency?

Your answer

Correct answer

B, which is line 23.

D, which is line 40.

```
 1    import os
 2    from flask import Flask, request, jsonify
 3    from flask_sqlalchemy import SQLAlchemy
 4    from sqlalchemy.exc import IntegrityError
 5    from flask_limiter import Limiter
 6    from flask_limiter.util import get_remote_address
 7    from werkzeug.security import generate_password_hash
 8    import logging
 9    from datetime import datetime
10    import threading
11
12    app = Flask(__name__)
A   13  app.config['SQLALCHEMY_DATABASE_URI'] = os.getenv('DATABASE_URL')
14    app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False
15
16    limiter = Limiter(app, key_func=get_remote_address)
17    db = SQLAlchemy(app)
18    logger = logging.getLogger(__name__)
19    _session_lock = threading.Lock()
20
21
22    class User(db.Model):
B   23      id = db.Column(db.Integer, primary_key=True)
24          username = db.Column(db.String(80), unique=True,
25                               nullable=False, index=True)
26          email = db.Column(db.String(120), unique=True, nullable=False, index=True)
```

```
27          password_hash = db.Column(db.String(128), nullable=False)
28          created_at = db.Column(db.DateTime, default=datetime.utcnow)
C 29        last_login = db.Column(db.DateTime)
30
31
32      @app.route('/api/v1/register', methods=['POST'])
33      @limiter.limit('5 per minute')
34      def register():
35          try:
36              username = request.json.get('username')
37              email = request.json.get('email')
38              password = request.json.get('password')
39
D 40            existing_user = User.query.filter_by(username=username).first()
41
42              if not existing_user:
43                  user = User(
44                      username=username,
45                      email=email,
46                      password_hash=generate_password_hash(password)
47                  )
48                  db.session.add(user)
49                  db.session.commit()
50
51                  logger.info('New user registered', extra={
52                      'username': username,
53                      'timestamp': datetime.utcnow().isoformat()
54                  })
55                  return jsonify({'status': 'success', 'message': 'Registration complete'}), 201
```

```
56
57            return jsonify({'status': 'error', 'message': 'Username taken'}), 409
58
59        except IntegrityError as e:
60            logger.error('Registration failed - integrity error', exc_info=True)
61            db.session.rollback()
62            return jsonify({'status': 'error', 'message': 'Username or email already exists'}), 409
63
64        except Exception as e:
65            logger.error('Registration failed - unexpected error', exc_info=True)
66            db.session.rollback()
67            return jsonify({'status': 'error', 'message': 'Registration failed'}), 500
68
```

Provide question feedback

QUESTION 3: (PYTHON) CRYPTOGRAPHIC FAILURES | USE OF A BROKEN OR RISKY CRYPTOGRAPHIC ALGORITHM

While reviewing a financial services platform's authentication system, you discover code that handles password hashing for user accounts. The system needs to securely store passwords for high-value trading accounts. The code below implements password hashing and verification. Select the secure replacement for the insecure line of code to implement proper cryptographic practices.

Your answer

D, which is: `password_hash = argon2.PasswordHasher().hash(credentials.password)`

Correct answer

D, which is: `password_hash = argon2.PasswordHasher().hash(credentials.password)`

Provide question feedback

QUESTION 4: (PYTHON) DATA PROCESSING ISSUES | INTEGER OVERFLOW OR WRAPAROUND

As a quantitative risk analyst for AlphaCapital (a systematic trading firm managing $50 billion in assets), you are reviewing the portfolio position sizing algorithm that processes large arrays of trade data using NumPy for high-performance calculations. The system handles thousands of trades per second. Identify the block of code containing an integer overflow vulnerability in NumPy array operations that could cause arithmetic wraparound, leading to incorrect position sizing calculations and significant financial losses.

Your answer

B, which is lines 14-17

Correct answer

B, which is lines 14-17.

```
    import sys
    import math
    import time
    import numpy as np
    from decimal import Decimal, getcontext
    from typing import List, Dict, Optional

A   class PositionSizingEngine:
        def __init__(self):
            self.max_position_size = np.int32(2147483647)
            self.risk_multiplier = np.int32(1000000)
            self.volatility_scaling = np.int32(10000)

B       def calculate_base_position_size(self, account_balance: int, risk_percentage: int, volatilit
            base_size = np.int32(account_balance * risk_percentage)
            adjusted_size = np.int32(base_size * volatility_factor * self.volatility_scaling)
            return np.minimum(adjusted_size, self.max_position_size)

C       def calculate_leverage_multiplier(self, base_position: np.int32, leverage_ratio: np.int32) -
```

```
            if base_position <= 0 or leverage_ratio <= 1:
                return base_position
            if base_position > self.max_position_size // leverage_ratio:
                return self.max_position_size
            leveraged_position = np.int32(base_position * leverage_ratio)
            return np.minimum(leveraged_position, self.max_position_size)

    D   def process_high_frequency_trade(self, account_balance: int, risk_pct: int, volatility: int,
            try:
                adjusted_position = self.calculate_base_position_size(account_balance, risk_pct, np.
                final_position = self.calculate_leverage_multiplier(adjusted_position, np.int32(leve
                return {
                    'base_position': int(np.int32((account_balance * risk_pct) // 100)),
                    'adjusted_position': int(adjusted_position),
                    'final_position': int(final_position),
                    'account_balance': account_balance
                }

            except Exception as e:
                print(f"Position calculation error: {e}")
                return {'error': str(e)}

def main():
    engine = PositionSizingEngine()

    account_balance = 500000000
    risk_percentage = 15
    volatility_factor = 2500
    leverage_ratio = 100
```

```
        result = engine.process_high_frequency_trade(
            account_balance, risk_percentage, volatility_factor, leverage_ratio
        )

        print(f"Trading Results: {result}")

    if __name__ == "__main__":
        main()

```

Provide question feedback

QUESTION 5: (PYTHON) DESIGN AND CONFIGURATION FLAWS | IMPROPER INPUT VALIDATION

During a security audit of a large-scale CI/CD platform used by thousands of development teams, you discover code that handles automated deployment pipelines. The system allows teams to provision and configure their own build environments. Which line introduces an input validation vulnerability that could lead to remote code execution or system compromise?

Your answer

B, which is line 57.

Correct answer

C, which is line 64.

```
1   import boto3
2   import json
3   from fastapi import FastAPI, HTTPException, Depends, Security
4   from fastapi.security import APIKeyHeader
5   from pydantic import BaseModel, validator
```

```
    import subprocess
    import logging
    from typing import Optional, List
    from datetime import datetime

    app = FastAPI(title="Build Environment Provisioner")
    ec2 = boto3.client('ec2')
    logger = logging.getLogger(__name__)
    api_key_header = APIKeyHeader(name="X-API-Key")

    class ResourceRequest(BaseModel):
        instance_type: str
        region: str
        script: str
        team_id: str
        environment: str

        @validator('instance_type')
        def validate_instance_type(cls, v):
            allowed_types = ['t2.micro', 't2.small', 't2.medium']
            if v not in allowed_types:
                raise ValueError(f'Instance type must be one of: {allowed_types}')
            return v

        @validator('environment')
        def validate_environment(cls, v):
            if v not in ['dev', 'staging', 'prod']:
                raise ValueError('Environment must be dev, staging, or prod')
            return v
```

```
35
36      def validate_region(region: str) -> bool:
37          valid_regions = ['us-east-1', 'us-west-2', 'eu-west-1']
38          return region in valid_regions
39
40      async def verify_team_access(team_id: str, api_key: str = Depends(api_key_header)):
41
42          if not is_team_authorized(team_id, api_key):
43              raise HTTPException(status_code=403, detail='Team not authorized')
44          return team_id
45
46      @app.post('/api/v1/provision')
47      async def provision_resource(
48          request: ResourceRequest,
49          team_id: str = Depends(verify_team_access)
50      ):
51          if not validate_region(request.region):
52              raise HTTPException(status_code=400, detail='Invalid region')
53
54          try:
55
56              timestamp = datetime.utcnow().strftime('%Y%m%d_%H%M%S')
B 57            script_path = f'/tmp/build_env_{request.team_id}_{timestamp}.sh'
58
59
60              with open(script_path, 'w') as f:
61                  f.write(request.script)
62
63
```

```
C   64          subprocess.run(['bash', script_path], shell=True)
    65
    66
D   67          response = ec2.run_instances(
    68              InstanceType=request.instance_type,
    69              MinCount=1,
    70              MaxCount=1,
    71              UserData=request.script,
    72              TagSpecifications=[{
    73                  'ResourceType': 'instance',
    74                  'Tags': [
    75                      {'Key': 'Team', 'Value': request.team_id},
    76                      {'Key': 'Environment', 'Value': request.environment}
    77                  ]
    78              }]
    79          )
    80
    81          instance_id = response['Instances'][0]['InstanceId']
    82          logger.info('Instance provisioned', extra={
    83              'team_id': request.team_id,
    84              'instance_id': instance_id,
    85              'environment': request.environment
    86          })
    87
    88          return {
    89              'instance_id': instance_id,
    90              'status': 'provisioning'
    91          }
    92
```

```
        except Exception as e:
            logger.error('Provisioning failed', exc_info=True)
            raise HTTPException(status_code=500, detail=str(e))
```

Provide question feedback

QUESTION 6: (PYTHON) IDENTIFICATION AND AUTHENTICATION FAILURES | WEAK PASSWORD RECOVERY MECHANISM

While reviewing a financial services platform's password recovery system, you discover code handling customer account resets. The system processes thousands of password resets daily across multiple regions. Which line introduces a critical authentication vulnerability that could allow unauthorized access?

Your answer

A, which is line 19.

Correct answer

A, which is line 19.

```
import logging
import secrets
import smtplib
from email.mime.text import MIMEText
from datetime import datetime
from typing import
from flask import Flask, request, jsonify

app = Flask(__name__)
logger = logging.getLogger(__name__)


class PasswordRecovery:
    def __init__(self):
        self.users = {}
```

```
        self.reset_tokens = {}

    def generate_reset_token(self) -> str:
        return ''.join(secrets.choice('0123456789') for _ in range(4))

    def clear_expired_tokens(self):
        now = datetime.utcnow()
        expired = [
            email for email, data in self.reset_tokens.items()
            if (now - data['created']).seconds > 3600
        ]
        for email in expired:
            del self.reset_tokens[email]

    def send_reset_email(self, email: str, token: str):
        try:
            msg = MIMEText(f'Your password reset code is: {token}')
            msg['Subject'] = 'Password Reset Request'
            msg['From'] = 'security@company.com'
            msg['To'] = email

            with smtplib.SMTP('smtp.company.com') as server:
                server.starttls()
                server.login('smtp_user', 'smtp_pass')
                server.send_message(msg)

            logger.info('Reset email sent', extra={'email': email})

        except Exception as e:
```

```
            logger.error('Failed to send email', exc_info=True)
            raise

    def request_reset(self, email: str) -> bool:
        if email not in self.users:
            logger.warning('If an account with that email exists, we have sent password reset ins
                           extra={'email': email})
            return False

        self.clear_expired_tokens()
        token = self.generate_reset_token()

        self.reset_tokens[email] = {
            'token': token,
            'attempts': 0,
            'created': datetime.utcnow()
        }

        self.send_reset_email(email, token)
        return True

    def verify_reset(self, email: str, token: str, new_password: str) -> bool:
        if email not in self.reset_tokens:
            return False

        token_data = self.reset_tokens[email]
        token_data['attempts'] += 1

        if token_data['attempts'] >= 3:
```

```
                del self.reset_tokens[email]
                logger.warning('Too many reset attempts',
                               extra={'email': email})
                return False

            if (datetime.utcnow() - token_data['created']).seconds > 3600:
                del self.reset_tokens[email]
                return False

            if token == token_data['token']:
                self.users[email]['password'] = new_password
                del self.reset_tokens[email]
                logger.info('Password reset successful',
                            extra={'email': email})
                return True

            return False


    recovery = PasswordRecovery()


    @app.route('/api/v1/password/reset', methods=['POST'])
    def request_reset():
        email = request.form.get('email')
        if not email:
            return jsonify({'error': 'Email required'}), 400

        if recovery.request_reset(email):
```

```
103             return jsonify({'status': 'Reset email sent'})
104         return jsonify({'error': 'Invalid email'}), 404
105
```

Provide question feedback

QUESTION 7: (PYTHON) BROKEN ACCESS CONTROL | IMPROPER ACCESS CONTROL

While reviewing a collaborative document management system's access control implementation, you discover code that handles document sharing and permissions. The system is used by organizations to manage sensitive documents with different access levels. Identify the block of code that could allow unauthorized access to documents due to improper access control.

Your answer

C, which is lines 44-69

Correct answer

C, which is lines 44-69.

```
1     from typing import Optional, Dict, List, Union
A 2   from fastapi import FastAPI, HTTPException, Security, Depends
3     from fastapi.security import OAuth2PasswordBearer
4     from pydantic import BaseModel, UUID4
5     import logging
6     from datetime import datetime
7     from enum import Enum
8
9     logger = logging.getLogger(__name__)
10
11    class AccessLevel(str, Enum):
12        READ = "read"
13        WRITE = "write"
```

```
14          ADMIN = "admin"
15
B 16    class DocumentPermission(BaseModel):
17          document_id: UUID4
18          user_id: UUID4
19          access_level: AccessLevel
20
21      class Document(BaseModel):
22          id: UUID4
23          title: str
24          content: str
25          owner_id: UUID4
26          created_at: datetime
27          updated_at: datetime
28
29      class DocumentService:
30          def __init__(self):
31              self.documents = {}
32              self.permissions = {}
33
34          def get_document(self, document_id: UUID4, user_id: UUID4) -> Optional[Document]:
35              document = self.documents.get(document_id)
36              if not document:
37                  return None
38
39              # Check if user is document owner
40              if document.owner_id == user_id:
41                  return document
42
```

```
            # Check if user has explicit permission
C           permission_key = f"{document_id}_{user_id}"
            if permission_key in self.permissions:
                return document

            return None

        def share_document(
            self,
            document_id: UUID4,
            owner_id: UUID4,
            user_id: UUID4,
            access_level: AccessLevel
        ) -> bool:
            document = self.documents.get(document_id)
            if not document:
                return False

            permission = DocumentPermission(
                document_id=document_id,
                user_id=user_id,
                access_level=access_level
            )
            permission_key = f"{document_id}_{user_id}"
            self.permissions[permission_key] = permission
            return True

    app = FastAPI(title="Document Management API")
    document_service = DocumentService()
```

```python
oauth2_scheme = OAuth2PasswordBearer(tokenUrl="token")

async def get_current_user(token: str = Depends(oauth2_scheme)) -> UUID4:
    try:
        # Token validation would go here
        user_id = UUID4()  # Simplified for example
        return user_id
    except Exception as e:
        logger.error(f"Authentication failed: {e}")
        raise HTTPException(
            status_code=401,
            detail="Invalid authentication credentials"
        )

@app.get("/api/v1/documents/{document_id}")
async def get_document(
    document_id: UUID4,
    current_user: UUID4 = Depends(get_current_user)
) -> Document:
    document = document_service.get_document(document_id, current_user)
    if not document:
        raise HTTPException(
            status_code=404,
            detail="Document not found or access denied"
        )
    return document

@app.post("/api/v1/documents/{document_id}/share")
async def share_document(
```

```
        document_id: UUID4,
        user_id: UUID4,
        access_level: AccessLevel,
        current_user: UUID4 = Depends(get_current_user)
    ) -> Dict[str, str]:
        success = document_service.share_document(
            document_id,
            current_user,
            user_id,
            access_level
        )
        if not success:
            raise HTTPException(
                status_code=404,
                detail="Document not found"
            )
        return {"status": "success"}
```

Provide question feedback

QUESTION 8: (PYTHON) INJECTION FLAWS | IMPROPER NEUTRALIZATION OF SPECIAL ELEMENTS USED IN A COMMAND ('COMMAND INJECTION')

While reviewing a cloud infrastructure monitoring platform's diagnostic API, you discover code handling network tests for customer environments. The system processes thousands of automated health checks daily and includes input validation. Which line introduces a Command Injection vulnerability despite the security measures?

Your answer

D, which is line 67.

Correct answer

D, which is line 67.

```
1    import logging
```

```
from flask import Flask, request, jsonify
from flask.views import MethodView
import subprocess
import re
from datetime import datetime

app = Flask(__name__)
logger = logging.getLogger(__name__)


class NetworkDiagnostics:
    ALLOWED_COMMANDS = {
        'ping': {'flag': '-c', 'max_count': 4},
        'traceroute': {'flag': '-m', 'max_count': 30},
        'nslookup': {'flag': '-timeout', 'max_count': 5}
    }

    @staticmethod
    def validate_hostname(host: str) -> bool:
        if not host or len(host) > 255:
            return False
        pattern = re.compile(r'^[a-zA-Z0-9][a-zA-Z0-9.-]*[a-zA-Z0-9]$')
        return bool(pattern.match(host))

    @staticmethod
    def validate_command(cmd: str) -> bool:
        return cmd in NetworkDiagnostics.ALLOWED_COMMANDS

    @staticmethod
```

```
C   31      def validate_count(count: int, cmd: str) -> bool:
    32          if not isinstance(count, int) or count < 1:
    33              return False
    34          return count <= NetworkDiagnostics.ALLOWED_COMMANDS[cmd]['max_count']
    35
    36
    37  class NetworkTestAPI(MethodView):
    38      def __init__(self):
    39          self.diagnostics = NetworkDiagnostics()
    40
    41      def get(self):
    42          cmd = request.args.get('command')
    43          host = request.args.get('host')
    44          count = request.args.get('count', 1, type=int)
    45
    46          if not cmd or not host:
    47              return jsonify({'error': 'Missing required parameters'}), 400
    48
    49          if not self.diagnostics.validate_command(cmd):
    50              logger.warning('Invalid command attempted', extra={
    51                  'command': cmd,
    52                  'remote_addr': request.remote_addr
    53              })
    54              return jsonify({'error': 'Invalid command'}), 400
    55
    56          if not self.diagnostics.validate_hostname(host):
    57              logger.warning('Invalid hostname', extra={
    58                  'hostname': host,
    59                  'remote_addr': request.remote_addr
```

```
60              })
61              return jsonify({'error': 'Invalid hostname'}), 400
62
63          if not self.diagnostics.validate_count(count, cmd):
64              return jsonify({'error': 'Invalid count parameter'}), 400
65
66          try:
D 67            command = f'{cmd} {self.diagnostics.ALLOWED_COMMANDS[cmd]["flag"]} {count} {host}'
68              output = subprocess.check_output(command, shell=True, timeout=30)
69
70              logger.info('Network test completed', extra={
71                  'command': cmd,
72                  'host': host,
73                  'count': count,
74                  'timestamp': datetime.utcnow().isoformat()
75              })
76
77              return jsonify({
78                  'status': 'success',
79                  'command': cmd,
80                  'output': output.decode('utf-8')
81              })
82
83          except subprocess.TimeoutExpired:
84              logger.error('Command timeout', extra={'command': command})
85              return jsonify({'error': 'Command timed out'}), 500
86
87          except subprocess.CalledProcessError as e:
88              logger.error('Command failed', extra={
```

```
                    'command': command,
                    'exit_code': e.returncode
                })
                return jsonify({'error': 'Command failed'}), 500

            except Exception as e:
                logger.error('Unexpected error', exc_info=True)
                return jsonify({'error': 'Internal error'}), 500


app.add_url_rule('/api/v1/network/test',
                 view_func=NetworkTestAPI.as_view('network_test'))

if __name__ == '__main__':
    app.run(debug=False, host='0.0.0.0', port=5000)

```

Provide question feedback

QUESTION 9: (PYTHON) INJECTION | IMPROPER NEUTRALIZATION OF SPECIAL ELEMENTS USED IN AN SQL COMMAND ('SQL INJECTION')

While reviewing a healthcare provider's patient records API, you discover code that handles patient search functionality. The system is used by medical staff to query patient information across multiple departments. Identify the block of code that could allow an attacker to perform SQL injection and access unauthorized patient records.

Your answer

B, which is lines 27-46

Correct answer

B, which is lines 27-46.

```
from typing import Optional, List, Dict
from fastapi import FastAPI, HTTPException, Depends
```

```python
from fastapi.security import APIKeyHeader
from sqlalchemy import create_engine, text
from sqlalchemy.exc import SQLAlchemyError
import logging
from datetime import datetime
from pydantic import BaseModel

logger = logging.getLogger(__name__)


class DatabaseConfig:
    DB_URL = "postgresql://user:pass@localhost:5432/healthcare"


class PatientSearchParams(BaseModel):
    name: Optional[str] = None
    department: Optional[str] = None
    admission_date: Optional[datetime] = None


class PatientService:
    def __init__(self):
        self.engine = create_engine(DatabaseConfig.DB_URL)

    def search_patients(self, params: PatientSearchParams) -> List[Dict]:
        query = "SELECT id, name, department, admission_date FROM patients WHERE 1=1"

        if params.name:
            query += f" AND name LIKE '%{params.name}%'"
```

```
32
33              if params.department:
34                  query += f" AND department = '{params.department}'"
35
36              if params.admission_date:
37                  query += f" AND DATE(admission_date) = '{params.admission_date.date()}'"
38
39              try:
40                  with self.engine.connect() as conn:
41                      result = conn.execute(text(query))
42                      return [dict(row) for row in result]
43
44              except SQLAlchemyError as e:
45                  logger.error(f"Database error: {e}")
46                  raise HTTPException(status_code=500, detail="Database error")
47
48
49      app = FastAPI(title="Healthcare Records API")
50      patient_service = PatientService()
51      api_key_header = APIKeyHeader(name="X-API-Key")
52
53
C     54  async def verify_api_key(api_key: str = Depends(api_key_header)) -> bool:
55          if not api_key == "YOUR-API-KEY":
56              raise HTTPException(status_code=403, detail="Invalid API key")
57          return True
58
59
D     60  @app.post("/api/v1/patients/search")
```

```python
    async def search_patients(
        params: PatientSearchParams,
        authenticated: bool = Depends(verify_api_key)
    ) -> List[Dict]:
        try:
            return patient_service.search_patients(params)
        except Exception as e:
            logger.error(f"Search failed: {e}")
            raise HTTPException(status_code=500, detail="Search failed")
```

Provide question feedback

QUESTION 10: (PYTHON) FILE UPLOAD SECURITY | INPUT VALIDATION

While reviewing a healthcare platform's access control system, you discover code that handles permission validation for accessing patient records. The system needs to verify user permissions and session expiration before allowing access to sensitive medical data. The code below implements the permission checking logic. For the insecure line of code, select the secure replacement that implements proper access control.

Your answer

C, which is: `return session and session.expires_at > datetime.now() and required_permission in session.permissions`

Correct answer

C, which is: `return session and session.expires_at > datetime.now() and required_permission in session.permissions`

Provide question feedback

QUESTION 11: (PYTHON) SECURITY LOGGING AND MONITORING FAILURES | EXPOSURE OF SENSITIVE INFORMATION IN LOG FILES

While reviewing a gaming platform's security logs, you discover code handling in-game purchases. The system processes millions of microtransactions daily across multiple game titles. Identify the code block that contains the

most critical security logging vulnerability.

Your answer

A, which is lines 11-32

Correct answer

A, which is lines 11-32.

```
1   import logging
2   import json
3   from datetime import datetime
4   from typing import Dict
5   from flask import Flask, request
6   import structlog
7
8   app = Flask(__name__)
9   logger = structlog.get_logger()
10
A 11  class PaymentLogger:
12      def __init__(self):
13          self.logger = logging.getLogger('payment_audit')
14          self.logger.setLevel(logging.INFO)
15          file_handler = logging.FileHandler('payments.log')
16          self.logger.addHandler(file_handler)
17
18      def log_transaction(self, payment_data: Dict):
19          """Log payment transaction details"""
20          try:
21              event_data = {
22                  'timestamp': datetime.utcnow().isoformat(),
23                  'user_id': payment_data['user_id'],
```

```
                    'amount': payment_data['amount'],
                    'card_number': payment_data['card_number'],
                    'cvv': payment_data['cvv'],
                    'expiry': payment_data['expiry']
                }
                self.logger.info('Payment processed',
                                 extra={'data': json.dumps(event_data)})
            except Exception:
                pass

B   class SecurityLogger:
        def __init__(self):
            self.logger = logging.getLogger('security_audit')
            self.logger.setLevel(logging.WARNING)
            file_handler = logging.FileHandler('security.log')
            self.logger.addHandler(file_handler)

        def log_failed_login(self, username: str):
            """Log failed login attempts"""
            try:
                self.logger.warning(f'Login failed for user: {username}')
            except Exception as e:
                print(f'Failed to log: {e}')

C   class AdminLogger:
        def __init__(self):
            self.logger = logging.getLogger('admin_audit')
            self.logger.setLevel(logging.INFO)
            file_handler = logging.FileHandler('admin.log')
```

```
53              self.logger.addHandler(file_handler)
54
55          def log_action(self, admin_id: str, action: str):
56              """Log administrative actions"""
57              try:
58                  self.logger.info(f'Admin {admin_id}: {action}')
59              except Exception as e:
60                  self.logger.error('Logging failed', extra={'error': str(e)})
61
D 62      class ErrorLogger:
63          def __init__(self):
64              self.logger = logging.getLogger('error_audit')
65              self.logger.setLevel(logging.ERROR)
66              file_handler = logging.FileHandler('errors.log')
67              self.logger.addHandler(file_handler)
68
69          def log_error(self, error: Exception):
70              """Log system errors"""
71              try:
72                  error_data = {
73                      'error': str(error),
74                      'traceback': error.__traceback__,
75                      'env': dict(request.environ),
76                      'headers': dict(request.headers)
77                  }
78                  self.logger.error('System error occurred',
79                                    extra={'error_details': json.dumps(error_data)})
80              except Exception as e:
81                  print(f'Error logging failed: {e}')
```

```
82
83      payment_logger = PaymentLogger()
84      security_logger = SecurityLogger()
85      admin_logger = AdminLogger()
86      error_logger = ErrorLogger()
```

Provide question feedback

QUESTION 12: (PYTHON) SECURITY MISCONFIGURATION | CONFIGURATION

While reviewing a cloud-based machine learning platform's configuration management system, you discover code that handles environment setup and API authentication. The system is used by data scientists to train and deploy models. Identify the block of code that could expose sensitive configuration data due to a security misconfiguration.

Your answer

B, which is lines 25-29

Correct answer

C, which is lines 36-50.

```
1       from typing import Optional, Dict, Union
2       from fastapi import FastAPI, HTTPException, Security
3       from fastapi.security import APIKeyHeader
4       from pydantic import BaseModel, SecretStr
5       import yaml
6       import logging
7       import json
8       from pathlib import Path
9
10      logger = logging.getLogger(__name__)
11
12      class ConfigManager:
```

```
13          def __init__(self, config_path: str):
14              self.config_path = Path(config_path)
15              self.config = self._load_config()
16
A 17        def _load_config(self) -> Dict:
18              try:
19                  with open(self.config_path) as f:
20                      return yaml.safe_load(f)
21              except Exception as e:
22                  logger.error(f"Failed to load config: {e}")
23                  return {}
24
B 25        def get_config(self, env: str = "development") -> Dict:
26              if env not in self.config:
27                  logger.warning(f"Environment {env} not found")
28                  return {}
29              return self.config[env]
30
31      class MLPlatform:
32          def __init__(self):
33              self.config = ConfigManager("/etc/mlplatform/config.yaml")
34              self.debug = True
35
C 36        def get_platform_info(self) -> Dict:
37              config = self.config.get_config()
38              if self.debug:
39                  return {
40                      "status": "running",
41                      "debug": True,
```

```
                    "config": config,
                    "api_key": config.get("api_key"),
                    "db_url": config.get("db_url"),
                    "aws_secret": config.get("aws_secret")
                }
            return {
                "status": "running",
                "version": "1.0.0"
            }

D   app = FastAPI(title="ML Platform API")
    platform = MLPlatform()

    @app.get("/api/v1/status")
    async def get_status() -> Dict:
        return platform.get_platform_info()

    @app.get("/api/v1/config")
    async def get_config(env: str = "development") -> Dict:
        return platform.config.get_config(env)

    if __name__ == "__main__":
        import uvicorn
        uvicorn.run(app, host="0.0.0.0", port=8000)
```

Provide question feedback

QUESTION 13: (PYTHON) SERVER-SIDE REQUEST FORGERY | SERVER-SIDE REQUEST FORGERY (SSRF)

While reviewing a cloud infrastructure management platform's resource fetcher, you discover code that handles AWS EC2 instance metadata retrieval. The system needs to securely fetch instance metadata for monitoring and configuration. The code below implements the metadata fetching logic. For the insecure line of code, select the secure replacement that prevents SSRF attacks.

Your answer

C, which is: `if path in self.ALLOWED_PATHS:`

Correct answer

C, which is: `if path in self.ALLOWED_PATHS:`

Provide question feedback

QUESTION 14: (PYTHON) SOFTWARE AND DATA INTEGRITY FAILURES | ML MODEL SUPPLY CHAIN SECURITY

As a security engineer for DataSync Corp (cloud data synchronization platform), identify the block of code containing a software and data integrity failure vulnerability. This vulnerability could allow arbitrary code execution or data manipulation.

Your answer

C, which is lines 83-104

Correct answer

C, which is lines 83-104.

```
1    from flask import Flask, request, jsonify
2    from werkzeug.security import check_password_hash, generate_password_hash
3    import pickle
4    import base64
5    import json
6    import hashlib
7    import secrets
8    import logging
9    import time
```

```python
from typing import Dict, Tuple, Optional, Any

logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s - %(name)s - %(levelname)s - %(message)s'
)
logger = logging.getLogger('datasync_api')

app = Flask(__name__)
app.config['SECRET_KEY'] = secrets.token_hex(32)
app.config['MAX_CONTENT_LENGTH'] = 10 * 1024 * 1024  # 10MB

class AuthenticationService:
    def __init__(self):
        self.user_database = {
            "admin": generate_password_hash("SecureAdmin#2024", method='pbkdf2:sha256:260000')
        }
        self.active_sessions: Dict[str, Dict[str, Any]] = {}

    def verify_credentials(self, username: str, password: str) -> bool:
        if not username or not password:
            return False
        if username in self.user_database:
            return check_password_hash(self.user_database[username], password)
        return False

    def create_session_token(self, username: str) -> str:
        session_token = secrets.token_urlsafe(32)
        self.active_sessions[session_token] = {
```

```
            "username": username,
            "created_at": time.time()
        }
        return session_token

    def verify_session(self, session_token: str) -> Tuple[bool, Optional[str]]:
        if not session_token:
            return False, None
        if session_token in self.active_sessions:
            return True, self.active_sessions[session_token]["username"]
        return False, None

class DataSyncProcessor:
    def serialize_to_json(self, data: Any) -> str:
        json_bytes = json.dumps(data, separators=(',', ':')).encode('utf-8')
        return base64.b64encode(json_bytes).decode('ascii')

    def deserialize_from_json(self, encoded_data: str) -> Any:
        decoded_bytes = base64.b64decode(encoded_data)
        return json.loads(decoded_bytes.decode('utf-8'))

    def calculate_integrity_hash(self, data: Any) -> str:
        data_string = json.dumps(data, sort_keys=True, separators=(',', ':'))
        return hashlib.sha256(data_string.encode('utf-8')).hexdigest()

auth_service = AuthenticationService()
sync_processor = DataSyncProcessor()

@app.route('/api/v1/data/sync', methods=['POST'])
```

```
68      def sync_data_endpoint():
69          try:
70              authorization_header = request.headers.get('Authorization', '')
71              if not authorization_header.startswith('Bearer '):
72                  return jsonify({"error": "Missing or invalid authorization"}), 401
73
74              token = authorization_header[7:]  # Remove 'Bearer ' prefix
75              is_valid, username = auth_service.verify_session(token)
76              if not is_valid:
77                  return jsonify({"error": "Session expired or invalid"}), 401
78
79              request_body = request.get_json()
80              if not request_body or 'payload' not in request_body:
81                  return jsonify({"error": "Missing payload in request"}), 400
82
C 83            encoded_payload = request_body['payload']
84
85              deserialized_data = pickle.loads(base64.b64decode(encoded_payload))
86
87              sync_metadata = {
88                  "username": username,
89                  "data": deserialized_data,
90                  "integrity_hash": sync_processor.calculate_integrity_hash(deserialized_data),
91                  "timestamp": time.strftime('%Y-%m-%d %H:%M:%S')
92              }
93
94              logger.info(f"Data synchronized for user: {username}, hash: {sync_metadata['integrity_h
95
96              return jsonify({
```

```
                "status": "success",
                "integrity_hash": sync_metadata["integrity_hash"],
                "timestamp": sync_metadata["timestamp"]
            }), 200

        except Exception as error:
            logger.error(f"Sync operation failed: {str(error)}", exc_info=True)
            return jsonify({"error": "Data synchronization failed"}), 500

@app.route('/api/v1/data/retrieve', methods=['GET'])
def retrieve_data_endpoint():
    authorization_header = request.headers.get('Authorization', '')
    if not authorization_header.startswith('Bearer '):
        return jsonify({"error": "Missing or invalid authorization"}), 401

    token = authorization_header[7:]
    is_valid, username = auth_service.verify_session(token)
    if not is_valid:
        return jsonify({"error": "Session expired or invalid"}), 401

    sample_dataset = {
        "owner": username,
        "documents": ["report_2024.pdf", "analysis.xlsx", "presentation.pptx"],
        "last_modified": "2024-01-15T10:30:00Z"
    }

    encoded_response = sync_processor.serialize_to_json(sample_dataset)

    return jsonify({
```

```
            "payload": encoded_response,
            "encoding": "base64_json"
        }), 200

    @app.route('/api/v1/auth/login', methods=['POST'])
    def authenticate_user():
        try:
            credentials = request.get_json()
            if not credentials:
                return jsonify({"error": "Invalid request format"}), 400

            username = credentials.get('username', '').strip()
            password = credentials.get('password', '')

            if not username or not password:
                return jsonify({"error": "Username and password required"}), 400

            if auth_service.verify_credentials(username, password):
                session_token = auth_service.create_session_token(username)
                return jsonify({
                    "token": session_token,
                    "token_type": "Bearer"
                }), 200

            return jsonify({"error": "Invalid username or password"}), 401

        except Exception as error:
            logger.error(f"Authentication failed: {str(error)}")
            return jsonify({"error": "Authentication service unavailable"}), 500
```

```
155
156     if __name__ == '__main__':
157         app.run(host='0.0.0.0', port=8080, debug=False)
```

Provide question feedback

QUESTION 15: (PYTHON) VULNERABLE AND OUTDATED COMPONENTS | LEGACY SSL USAGE

As a DevSecOps engineer for a healthcare AI platform, identify the critical version pinning vulnerability in this Python machine learning system. This vulnerability could allow automatic updates to compromised dependency versions and expose patient health data.

Your answer

B, which is lines 24-27

Correct answer

B, which is lines 24-27.

```
1     import pandas as pd
2     import numpy as np
3     import tensorflow as tf
4     from sklearn.model_selection import train_test_split
5     from sklearn.preprocessing import StandardScaler
6     import joblib
7     import logging
8     from datetime import datetime
9     import os
10    import requests
11    import json
12
13    # requirements.txt shows versions:
14    """
A 15  tensorflow==2.13.0
16    scikit-learn==1.3.0
```

```
numpy==1.24.3
joblib==1.3.1
flask==2.3.2
gunicorn==20.1.0
psycopg2-binary==2.9.6
redis==4.5.5

pandas
requests
sqlalchemy
celery
"""

class HealthcareMLPipeline:
    def __init__(self, model_path="./models/"):
        self.model_path = model_path
        self.scaler = None
        self.model = None
        self.logger = self._setup_logging()

    def _setup_logging(self):
        logger = logging.getLogger('healthcare_ml')
        logger.setLevel(logging.INFO)
        handler = logging.FileHandler('ml_predictions.log')
        formatter = logging.Formatter('%(asctime)s - %(levelname)s - %(message)s')
        handler.setFormatter(formatter)
        logger.addHandler(handler)
        return logger

```

```
D  46      def load_patient_data(self, data_source):
47              """Load patient data from secure healthcare database"""
48              try:
49                  # Load sensitive patient data
50                  df = pd.read_csv(data_source)
51                  self.logger.info(f"Loaded {len(df)} patient records")
52                  return df
53              except Exception as e:
54                  self.logger.error(f"Failed to load patient data: {str(e)}")
55                  return None
56
57          def preprocess_features(self, data):
58              """Preprocess medical features for ML model"""
59              # Remove patient identifiers but keep medical data
60              feature_columns = ['age', 'blood_pressure', 'cholesterol', 'glucose',
61                                 'heart_rate', 'bmi', 'family_history']
62
63              X = data[feature_columns]
64              y = data['diagnosis_risk']
65
66              # Split data
67              X_train, X_test, y_train, y_test = train_test_split(
68                  X, y, test_size=0.2, random_state=42
69              )
70
71              # Scale features
72              self.scaler = StandardScaler()
73              X_train_scaled = self.scaler.fit_transform(X_train)
74              X_test_scaled = self.scaler.transform(X_test)
```

```
75
76              return X_train_scaled, X_test_scaled, y_train, y_test
```

Provide question feedback